SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: COMPUTER NETWORKS
COURSE CODE: CSA07

COURSE OUTCOME COVERED

CO5: Measure network performance using key metrics with simulation tools. *(BL4,BL5)*

Assessment Tool Weightage: 30%

Annexure A

Pseudocode Writing Task (Algorithm Design)

1. In a network simulation using Cisco Packet Tracer, a student records total data received as 8000 bits in 4 seconds and observes that 200 packets were sent while 180 packets were received. Write a pseudocode algorithm to calculate the network throughput in bits per second and the packet loss percentage based on the given values. The algorithm should include validation to ensure that time and total packets sent are not zero to avoid errors. It should display both the throughput and packet loss results clearly. Add logic to classify the network as Efficient if throughput is greater than 1500 bps and packet loss is less than 10%, otherwise classify it as Inefficient. Finally, ensure the algorithm outputs a complete summary of the network performance based on the calculated metrics.
2. An organization has multiple network links connecting its branches. The administrator wants to monitor bandwidth utilization and identify overloaded links. Write a pseudocode algorithm that periodically collects the bandwidth usage of each network link and stores the values in an array or list. Use a loop to continuously monitor all links. Add conditions to detect when bandwidth utilization exceeds 80% and recommend switching traffic to a backup link. Explain how this algorithm supports efficient bandwidth management and prevents network congestion.
3. A company has six branch offices connected through multiple network links. Each link has different bandwidth, delay, and utilization values, which change throughout the day. The network administrator wants to continuously identify the best communication path between the headquarters and each branch based

on overall link quality rather than just the shortest distance.Write a pseudocode algorithm that periodically collects the bandwidth, delay, and utilization of every network link and stores the values in suitable arrays or lists. Use a loop to repeat the monitoring process at regular intervals. Calculate a link quality score for each path using the collected metrics and select the path with the highest score. Include conditions to detect when a link becomes overloaded or fails, automatically selecting an alternate path and generating an alert for the administrator. Explain how your algorithm improves network reliability, routing efficiency, and fault tolerance while adapting to changing network conditions.

Question 1: Network Throughput & Loss Analysis

```
ALGORITHM NetworkPerformanceEvaluator
BEGIN
   // Step 1: Initialize input variables based on network observations
   DECLARE total_data_bits AS INTEGER = 8000
   DECLARE total_time_seconds AS INTEGER = 4
   DECLARE packets_sent AS INTEGER = 200
   DECLARE packets_received AS INTEGER = 180

   DECLARE throughput AS REAL
   DECLARE packet_loss_pct AS REAL
   DECLARE status AS STRING

   // Step 2: Input Validation (Prevent division by zero)
   IF total_time_seconds <= 0 THEN
      PRINT "Error: Time duration must be greater than zero."
      EXIT
   ENDIF

   IF packets_sent <= 0 THEN
      PRINT "Error: Total packets sent must be greater than zero."
      EXIT
   ENDIF

   IF packets_received > packets_sent THEN
      PRINT "Error: Packets received cannot exceed packets sent."
      EXIT
   ENDIF
```

```
    // Step 3: Perform calculations
    // Throughput formula: Total Bits / Total Time (seconds)
    SET throughput = total_data_bits / total_time_seconds

    // Packet Loss Percentage formula: ((Sent - Received) / Sent) * 100
    SET packet_loss_pct = ((packets_sent - packets_received) / packets_sent) * 100.0

    // Step 4: Network Classification
    IF throughput > 1500 AND packet_loss_pct < 10 THEN
       SET status = "Efficient"
    ELSE
       SET status = "Inefficient"
    ENDIF

    // Step 5: Output complete performance summary
    PRINT "========================================="
    PRINT "      NETWORK PERFORMANCE SUMMARY       "
    PRINT "========================================="
    PRINT "Total Data Received : " + total_data_bits + " bits"
    PRINT "Time Duration       : " + total_time_seconds + " seconds"
    PRINT "Packets Sent        : " + packets_sent
    PRINT "Packets Received    : " + packets_received
    PRINT "-----------------------------------------"
    PRINT "Calculated Throughput: " + throughput + " bps"
    PRINT "Packet Loss Rate    : " + packet_loss_pct + "%"
    PRINT "Network Status      : " + status
    PRINT "========================================="

END
```

Walkthrough of Given Sample Data

- **Throughput Calculation:** $\frac{8000 \text{ bits}}{4 \text{ seconds}} = 2000 \text{ bps}$
- **Packet Loss Calculation:** $\frac{200-180}{200} \times 100 = \frac{20}{200} \times 100 = 10\%$
- **Evaluation:** Throughput (2000 bps) is greater than 1500 bps, but Packet Loss (10%) is **not** strictly less than 10%. As a result, the sample data classifies as **Inefficient**.

Question 2: Dynamic Bandwidth Overload Detection

```
ALGORITHM BandwidthMonitor
BEGIN
    // Step 1: Initialize link records and constants
    DECLARE num_links AS INTEGER = 8
    DECLARE link_names[8] AS STRING = ["Link_A", "Link_B", "Link_C", "Link_D", "Link_E", "Link_F", "Link_G", "Link_H"]
    DECLARE link_capacities[8] AS REAL = [100.0, 1000.0, 500.0, 250.0, 1000.0, 100.0, 500.0, 250.0] // in Mbps
    DECLARE backup_links[8] AS STRING = ["Link_E", "Link_A", "Link_F", "Link_H", "Link_B", "Link_C", "Link_D", "Link_G"]

    DECLARE current_usage[8] AS REAL
    DECLARE utilization_pct[8] AS REAL

    // Step 2: Continuous monitoring loop
    WHILE TRUE DO
        FOR i FROM 1 TO num_links DO
            // Collect live traffic usage (Mbps)
            SET current_usage[i] = FETCH_BANDWIDTH_USAGE(link_names[i])

            // Calculate percentage utilization
            SET utilization_pct[i] = (current_usage[i] / link_capacities[i]) * 100.0

            // Step 3: Threshold Evaluation
            IF utilization_pct[i] > 80.0 THEN
                PRINT "CRITICAL WARNING: " + link_names[i] + " is at " + utilization_pct[i] + "% capacity."
                PRINT "ACTION: Recommend shifting traffic immediately to backup link: " + backup_links[i]
            ELSE
                PRINT link_names[i] + " operating normally at " + utilization_pct[i] + "%."
            ENDIF
        ENDFOR

        WAIT_SECONDS(60) // Check interval
    ENDWHILE
END
```

How This Supports Management and Congestion Prevention

- **Proactive Thresholding:** By triggering warnings at 80% instead of waiting for 100% saturation, administrators can offload traffic before buffers fill and packet drops begin.
- **Load Redistribution:** Recommending predefined backup links ensures traffic finds an alternate path instantly, preventing severe network bottlenecks.

- **Visibility:** Storing metrics in arrays lets administrators spot usage trends over time, helping with capacity planning.

Question 3: Multi-Metric Dynamic Path Routing

```
ALGORITHM DynamicPathOptimizer
BEGIN
   DECLARE num_branches AS INTEGER = 6
   DECLARE num_links AS INTEGER = 15 // Total available connections across network

   DECLARE link_bandwidth[num_links] AS REAL
   DECLARE link_delay[num_links] AS REAL
   DECLARE link_utilization[num_links] AS REAL
   DECLARE link_status[num_links] AS STRING // "UP" or "DOWN"
   DECLARE link_score[num_links] AS REAL

   WHILE TRUE DO
      // Step 1: Collect metrics for all links
      FOR i FROM 1 TO num_links DO
         SET link_bandwidth[i] = SNMP_GET_BANDWIDTH(i)
         SET link_delay[i] = SNMP_GET_DELAY(i)      // in milliseconds
         SET link_utilization[i] = SNMP_GET_UTIL(i) // percentage 0-100
         SET link_status[i] = PING_CHECK(i)         // "UP" or "DOWN"

         // Step 2: Calculate Link Quality Score
         IF link_status[i] == "DOWN" OR link_utilization[i] >= 95.0 THEN
            SET link_score[i] = -1 // Disqualify failed or critically choked links
         ELSE
            // Higher score = higher bandwidth, lower delay, lower utilization
            SET link_score[i] = (link_bandwidth[i] * (1.0 - (link_utilization[i] / 100.0))) / (link_delay[i] +
1.0)
         ENDIF
      ENDFOR

      // Step 3: Assign best path for each branch office
      FOR branch FROM 1 TO num_branches DO
         DECLARE best_link AS INTEGER = -1
         DECLARE max_score AS REAL = -999.0

         FOR i FROM 1 TO num_links DO
            IF LINK_CONNECTS_BRANCH(i, branch) AND link_score[i] > max_score THEN
               SET max_score = link_score[i]
               SET best_link = i
            ENDIF
```

```
        ENDFOR

        // Step 4: Handle failures and route updates
        IF best_link != -1 THEN
            UPDATE_ROUTING_TABLE(branch, best_link)
        ELSE
            TRIGGER_ADMIN_ALERT("CRITICAL: Branch " + branch + " has no active viable paths!")
        ENDIF
      ENDFOR

      WAIT_SECONDS(30) // Continuous real-time optimization interval
   ENDWHILE
END
```

The mathematical link quality score formula used in the algorithm is:

$$Score_i = \frac{B_i \times (1 - U_i)}{D_i + 1}$$

where $B_i$is available bandwidth, $U_i$is the utilization fraction ($0 \leq U_i \leq 1$), and $D_i$is propagation delay.

How This Enhances Network Performance

- **Network Reliability:** Continuous real-time metric tracking ensures communication doesn't rely on static, potentially degraded routes.
- **Routing Efficiency:** Instead of blindly selecting the path with the fewest hops, it evaluates multi-variable conditions (bandwidth, delay, congestion) to pick the highest-performing pipeline.
- **Fault Tolerance:** If a link drops or spikes past 95%utilization, it is automatically disqualified, zeroing out routing to it while instantly triggering alerts and shifting traffic to a healthy alternative.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient